

Software: conocimiento representado en varios niveles de abstracción, siendo el mas detallado en una computadora. Una de las problemáticas del software es el nivel que ocupa en una solución (que tan automatizado o no esta). Va a evolucionando de una idea hasta un nivel mas fino que es la interpretación de 0 y 1 del hardware. NO ES SOLO CÓDIGO, es todo lo que implica la creación de ese código. Conjunto de programas procedimientos, reglas.

Frederick Brooks – No Silver Bullets & The Mythical ManMonth.

En “No Silver Bullets” propone dividir las características del software en El Accidente (la representación) que refieren a las dificultades que acompañan a la producción, pero no son inherentes (sintaxis compleja, limitaciones de memoria, tiempos de compilación, etc.) y la esencia (el concepto) que refiere a la construcción de estructuras conceptuales entrelazadas altamente complejas (algoritmos, flujos de datos y relaciones). El accidente fue casi resuelto, mientras que la esencia es lo que queda hoy. Escribir código no es el problema real. La parte física es la especificación, el diseño y la validación de la estructura conceptual.

La esencia del software complejo tiene cuatro propiedades irreducibles:

1. Complejidad: escalar software aumenta los elementos interactuando de forma no lineal. Todas las características deben estar bien pensadas.
2. Invisibilidad: el software no se ve, no está sujeto a las leyes de la física. Carece de representación espacial. Da la impresión de que como no se ve es fácil de modificar.
3. Conformidad: debe adaptarse a interfaces humanas arbitrarias y sistemas sin principios unificadores, dictados por el entorno.
4. Mutabilidad: es pura materia de pensamiento, infinitamente maleable y concretamente sometida a la presión del cambio continuo. Para dejar contento al cliente/usuario se tiene que concebir para un entorno que lo va a aceptar, somos una profesión de servicios y eso es lo mas complejo.

Hacer software es un proceso social y el problema mas grave es que las personas logren comunicarse correctamente. La etapa de requerimientos es la que aún da problemas, ya que decidir que se quiere hacer es indispensable. Incorporar cambios que impactan en la arquitectura es grande (segundo gran impacto luego de los requerimientos). Si no somos capaces de entender el fenómeno en el que trabajamos, difícilmente podremos cumplir con los objetivos propuestos.

La esencia del software resiste la automatización porque posee características únicas en la ingeniería humana.

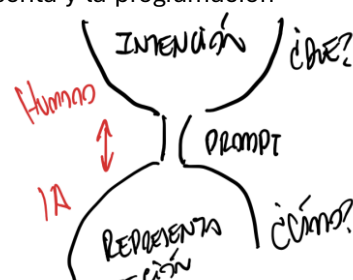
La historia del software es la eliminación sistemática de lo accidental. Los mayores saltos de productividad históricos no alteraron la esencia, simplemente eliminaron barreras artificiales. Rusia los grandes saltos atacaron exclusivamente el esfuerzo accidental. Tenemos la tendencia de que aparece algo y se supone que es “la bala de plata” que va a resolver los problemas del software.

- 60s: lenguajes de alto nivel. Se logro un factor de x5 de productividad. Se libero al programador de lidiar con bits, dígitos y registros + hardware.
- 70s: time-sharing. Se logro la inmediatez
- 80s/90s: herramientas unificadas. Facilita la interconexión de programas y formatos.

Apareció SCRUM: marco de trabajo para gestionar proyectos (no es una metodología y mucho menos una bala de plata). Actualmente la supuesta bala de plata es la IA. El cuello de botella es humano y se trata de decidir el “¿Qué?”, es decir, decidir que decir. La IA no te garantiza resolver el problema correcto, esto se debe a que la idea del producto es mental, pasa por un Prompt que pasaría a ser un nuevo lenguaje de alto nivel, que debe especificar con precisión matemática restricciones, estados y lógicas de negocio. El limite fundamental de esto es que, si el humano no comprende la complejidad esencial, el primer será incompleto, por lo que la IA no deduce la intención no descrita y la programación automática solo función en dominios con parámetros altamente restringidos. La IA ayuda muchísimo en lo accidental, pero no en lo esencial (software que realmente cumpla con los requerimientos). Ayuda en el como, no en el que.

Spec-Driven Development: Desarrollo orientado a los requerimientos.

El embudo de la especificación: el ancho de banda de la ejecución de la IA es infinito; el ancho de banda conceptual del humano es el límite físico. La intención (el Qué) es la idea del negocio, reglas



difusas, integraciones jefazo y requerimientos de usuario; el prompt es la traducción de ideas vagas a restricciones formales requiere esfuerzo puro. La representación es el como. El flujo total del sistema sigue estrangulado por la capacidad humana, el diseño conceptual, sin importar cuán rápido se genere el código.

Un proyecto es una unidad de gestión. No somos solo nosotros los que usamos proyectos, otras disciplinas también lo usan (industria automotriz, proyectos arquitectónicos), pero la forma de gestión de proyectos es muy distinta.

Un correcto desarrollo de un proyecto se da por el involucramiento del usuario, apoyo de la gerencia, enunciado claro de los requerimientos, planeamiento adecuado, expectativas realistas, hitos intermedios y personas involucradas competentes (40% del software que NO se tira).

Cuando no se hace lo mencionado (60% de los casos), se debe por los requerimientos incompletos, falta de involucramiento del usuario, falta de recursos, expectativas poco realistas, falta de apoyo de gerencia y requerimientos cambiantes.

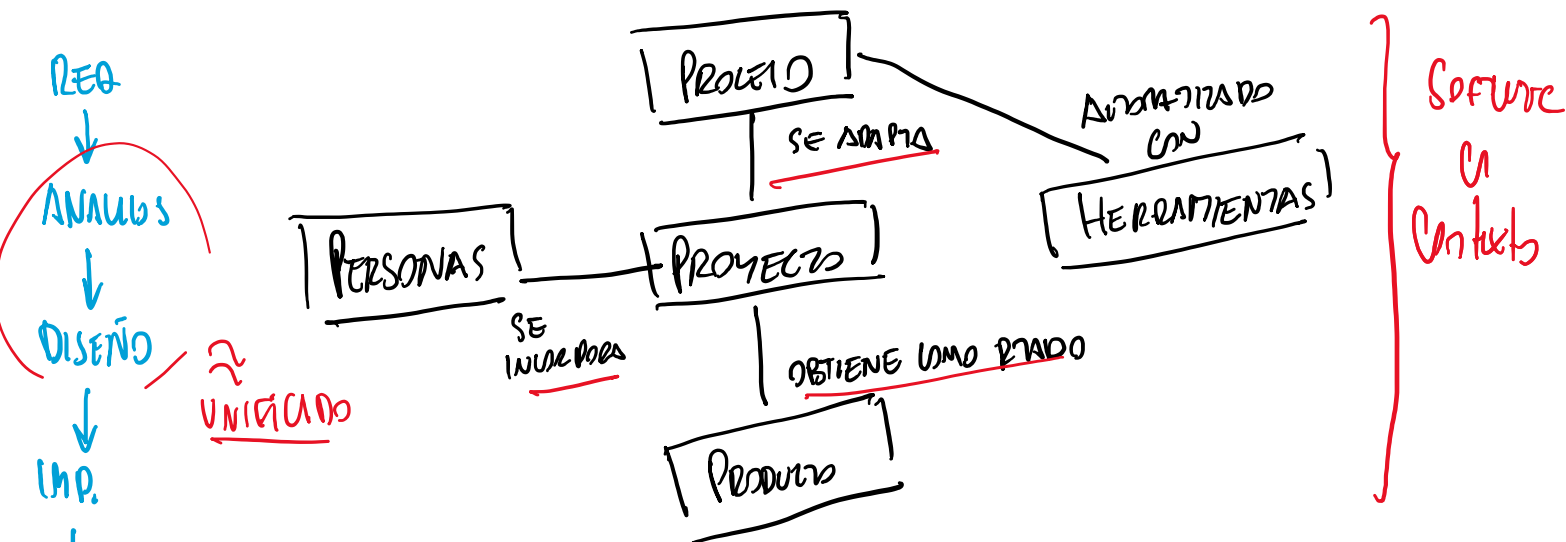
SABER PROGRAMAR NO ES SUFICIENTE. Como tampoco es correcto afirmar que es posible no saber programar.

Ingeniería: conjunto de herramientas y métodos que encaran una problemática de manera sistemática y organizada.

Ingeniería de software: disciplina de la ingeniería que se ocupa de todos los aspectos de la producción de software, desde la concepción inicial del sistema hasta su operación y mantenimiento.

SEBOK (Cuerpo de conocimientos de la ingeniería de software): corazón del diseño de la carrera y aquellas relacionadas al software. Está organizado en distintas disciplinas, pero en la materia nos centramos en disciplinas técnicas (Requerimientos, análisis y diseño, relacionadas directamente a crear el software como producto, **prueba y despliegue son las que se tomarán en cuenta**), disciplinas de gestión/Project Management (planificación de proyecto, monitoreo y control ya sea permanente o mediante hitos) y las disciplinas de soporte/protectoras/Transversales (Gestión de configuración de software, aseguramiento de charlas, métricas).

SOFTWARE CONFIGURATION MANAGEMENT (SCM): más allá del nombre, update. Esta disciplina no es lo mismo que usar una herramienta de versionado.



La disciplina de la ingeniería de software se mueve en estos cuatro horizontes. Un proceso de desarrollo de software te va a describir todas las etapas del ciclo de desarrollo de software. El PUD se adapta a las herramientas. Hay dos tipos de procesos; los procesos definidos y tenemos los empíricos. Dentro de los empíricos encontramos lean y ágil, ya que estos basan sus planteos en los empíricos. En los definidos encontramos algunos más tradicionales (el PUD es un tipo de proceso definido. SCRUM es un referente de ágil, que a su vez se basa en procesos empíricos. KAMBAN se adhiere a la filosofía Lean y también se basa en procesos empíricos.

El empirismo es experimental y se basa en aprender en base a los resultados de las experiencias, es decir que se le da poder al equipo para que decida y tenga retroalimentación rápida para mejorar lo que tenga que mejorar. Al contrario que los definidos, ya que estos se adaptan un poco a las necesidades, definiendo un proceso escrito para procesos asociados puedan tener previsibilidad (si usamos un proceso de la misma manera, los tiempos serán similares en cada iteración, los

resultados también, etc.). El problema de los definidos es que asumen que la experiencia es extrapolable a otros equipos (métricas incorrectas). Los procesos empíricos difieren en esto, ya que buscan el constante feedback y mejora continua.

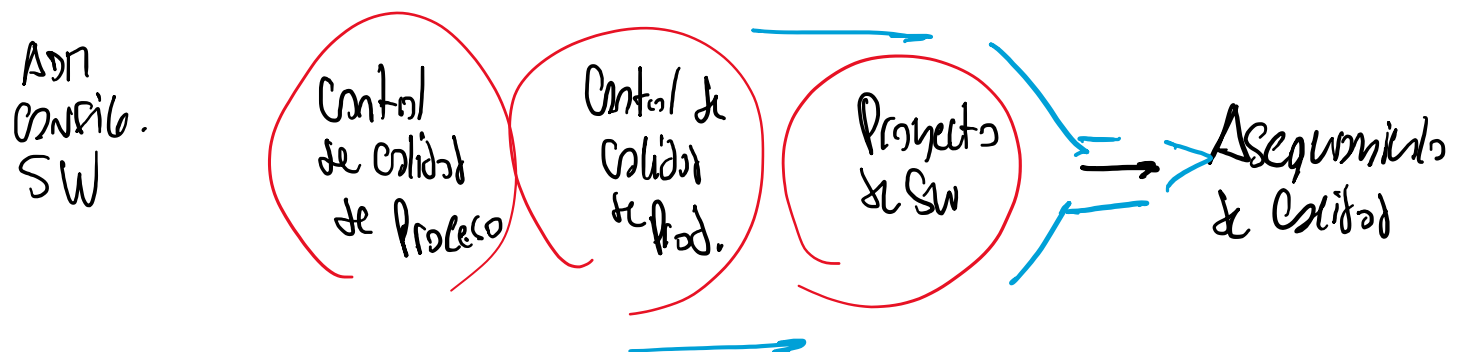
Adaptar parte de la base en como el conocimiento teórico se lleva al proyecto y se analiza (personas, perfiles, espacios, tecnologías). El producto que voy a construir también condiciona el tipo de proceso y su definición (sacar o justificar porque se considera o no cierta etapa). El proceso de ADAPTA al Proyecto.

El proyecto es la unidad de gestión del trabajo de la gente y administración de recursos para obtener como resultado el producto software. Las personas son el corazón del proyecto, ya que sin ellas no hay proyecto (no solo lo que participan activamente en el desarrollo, sino también clientes y usuarios, stakeholders).

Los procesos se definen en términos de roles para darle flexibilidad. Las herramientas automatizan lo mas que se pueden el proceso. Se debe adaptar el proyecto y proceso al contexto.

La Gestión de configuración de software busca resolver el problema de la integridad del software. La configuración de software es como una foto de todos los ítems de configuración guardados en un repositorio. Iguala el concepto software a ítem de configuración (software de interés para administrar). Toda pieza de interés para administrar en un proyecto es un ítem de configuración (CI- CONFIGURATION ÍTEM). Para que sea un ítem de configuración, tiene que poder guardarse en una computadora. Si se trabaja con prototipos en papel, es software, pero no es un ítem de configuración en ese formato. (Se debe digitalizar). Un ítem de configuración puede ser muchas cosas. Cada ítem de configuración tiene que tener un nombre único y una versión (se identifica unívocamente). Debemos tener la posibilidad de regresar a sus versiones anteriores. Para ello utilizamos herramientas de versionado. Esto ataca directamente al problema de la maleabilidad.

Los cambios del software tienen su origen en los cambios de negocio, cambios de requerimientos, soporte de cambios de productos asociados, reorganización de prioridades de la empresa por crecimiento, deuda técnica, cambios de presupuesto, etc. Es una disciplina transversal el soporte del software. Las disciplinas del soporte se debe tener una buena configuración de software.



Según la **IEEE**, la **gestión de configuración de software** (Software Configuration Management, SCM) puede definirse formalmente como:

La disciplina que aplica dirección y vigilancia técnica y administrativa para identificar y documentar las características funcionales y físicas de un elemento de configuración, controlar los cambios de esas características, registrar e informar el estado del procesamiento de los cambios y de la implementación, y verificar el cumplimiento de los requisitos especificados.

Debemos gestionar la configuración de software para mantener la integridad de un producto de software. Para que producto pueda tener integridad debe poder ser identificable en un momento, controlar sus cambios y mantener su integridad y origen. Un producto tiene integridad cuando se satisfacen las necesidades del usuario, puede ser fácil y completamente rastreado durante su ciclo de vida, satisface criterios de performance y cumple con sus expectativas de costo. No todos los Ítems de configuración tienen el mismo ciclo de vida.

Problema con el manejo de componentes: pérdida de componente, perdida de cambios, sincronía fuente-objeto-ejecutable, regresión de fallas, doble mantenimiento, superposición de cambios y cambios no validados.